5-2 · Docker Compose로 Spark 마스터-워커 구축 · 실습 1H

spark-submit으로 클러스터에 잡 제출

local[*] 이 spark://... 로 바뀌는 순간

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 마스터 컨테이너의 spark-submit 으로 pi.py 예제 제출

② first_job.py 를 만들어 넣고 master 가 spark:// 인지 확인

③ --total-executor-cores 2 로 제한해 UI 의 Cores 4 · 2 비교

④ 8g 를 요구해 WAITING 을 만들고, 두 잡을 동시에 돌리기

실행 중 마스터 UI 의 애플리케이션 이름을 누르면 5-1 의 Jobs·Stages 화면이 열립니다

실습 흐름

다섯 단계 — 예제 · 내 스크립트 · 자원 · 실패 · 동시

모든 제출은 마스터 컨테이너 안의 spark-submit 으로 — 6단계는 미니 과제

1. pi.py	2. first_job.py	3. 자원 제한	4. 과한 요구(실패)	5. 동시 제출
예제 제출 Pi is roughly	히어독 · cp 결과 세 줄	cores 2 Cores 4 vs 2	memory 8g WAITING	exec -d 로 두 개

핵심 명령

spark-submit 에 붙이는 것들

마스터 주소	--master spark://spark-master:7077
예제	/opt/spark/examples/src/main/python/pi.py 10
스크립트 만들기	cat > first_job.py << 'EOF' … EOF
파일 넣기	docker compose cp first_job.py spark-master:/tmp/first_job.py
결과 세 줄만	2>&1 | grep -E "^(count|master|executors)"
자원 제한	--total-executor-cores 2 --executor-memory 1g
일부러 실패	exec spark-master timeout 40 … --executor-memory 8g
동시 제출	docker compose exec -d spark-master …

제출 앞머리는 docker compose exec spark-master /opt/spark/bin/spark-submit

1·2단계

제출하고 로그를 읽는다 — 네 가지 증거

1. **pi.py — Pi is roughly 3.14...**
   뒷자리는 매번 다름(실측 3.141400 · 3.142680) · UI 에 PythonPi Cores 4 · FINISHED

2. **first_job.py — count = 500000**
   range(1, 1_000_001) 을 2배 해 1000000 초과만 센 결과

3. **master: spark://spark-master:7077**
   5-1 에서는 local[*] 이었던 자리 — 진짜 클러스터에서 돈다는 증거

4. **executors: 3**
   워커마다 익스큐터 하나씩 2개 + 드라이버 1개(이 함수는 드라이버도 센다)

3단계

자원 옵션을 바꿔 Cores 열을 비교

제한 없음 — 2단계 제출

워커 둘의 코어를 모두 쓴다

- Completed 의 first-cluster-job
- Cores 4
- Duration 실측 14초

VS

--total-executor-cores 2

--executor-memory 1g 와 함께 제한

- 같은 결과 세 줄
- Cores 2
- Duration 실측 17초 — 비슷하다

4단계 — 일부러 실패시키기

워커(2g)보다 큰 메모리를 요구한다

이론의 「자원이 모자라면 대기 상태로 멈춘다」를 직접 봅니다

memory 8g 요구	WARN 경고	마스터 UI	timeout 40	Completed
워커는 2g 배치 불가	15초 간격 두 줄	WAITING Cores 0 · 8.0 GiB	40초 뒤 프롬프트로	Cores 0 으로 옮겨 감

4단계 — 일부러 실패시키기

경고를 만나면 — 오류가 아니라 「기다리는 중」

1. **마스터 UI 에서 워커가 살아 있는지**
   Workers 표가 비어 있으면 자원을 줄 곳이 없다

2. **남은 코어·메모리가 요청보다 큰지**
   --executor-memory 가 워커 메모리(2g)보다 크면 배치되지 않는다

3. **다른 잡이 자원을 다 쓰고 있지는 않은지**
   먼저 제출된 잡이 전부 차지했으면 끝날 때까지 기다린다

로그만 보면

멈춘 것처럼 보인다

5단계

두 잡을 동시에 — 코어를 2개씩 나눈다

--total-executor-cores 2 씩

각 10초 안팎 · 둘 다 Cores 2



VS

옵션을 빼고 제출하면

첫 잡이 Cores 4 로 전부 차지한다



확인 포인트

이 1시간이 끝날 때 확인할 것

1. pi.py 로그 끝에 Pi is roughly 3.14... · UI 에 Cores 4
2. 결과 세 줄 count = 500000 · master spark:// · executors 3
3. Completed 의 first-cluster-job 두 줄 Cores 4 와 2
4. 8g 요구는 경고 15초 간격 두 줄 · UI 에 WAITING · Cores 0
5. 40초 뒤 timeout 으로 끊기고 Completed 로 옮겨 간다
6. exec -d 두 개가 동시에 RUNNING · Cores 2

5-2 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

클러스터 명령을 한 장으로 정리하고, 5-1 과 5-2 의 차이를 표로 비교한 뒤 5-3 을 위한 상태를 확인합니다

- ✓ 명령 요약 여덟 줄
- ✓ 5-1(한 대) vs 5-2(클러스터)
- ✓ 자주 겪는 오류 네 가지
- ✓ 5-3 준비 — 셋 Up · 마스터 UI 200

가져갈 것

UI 의 Completed 목록과 Cores 값